

Day04 Part C 学习文档 v1.0 : Runtime

如何思考 (How Runtime Thinks) - Context Builder Projection

本文是《从零实现 Agent Runtime》学习阶段的 Day04 Part C 正式学习文档。

Part A 建立了“Prompt 不是 Context”的世界观；Part B-01 解释 Runtime State 是什么；Part B-02 解释 Runtime State 如何创建、演化、持久化和恢复。Part C 继续回答一个关键问题：Runtime 如何把完整、复杂、内部化的 Runtime State，投影成 LLM 当前能够理解和推理的 Provider Messages。

与上一节的联系

Part B-02 的核心结论是：

Runtime 不只是维护一个 State，而是在编排一个不断创建、恢复、演化、持久化、销毁，并且可以在崩溃后恢复继续执行的 State 生命周期。

但是 Runtime State 本身不能直接发送给 LLM。

因为 Runtime State 是 Runtime 的完整世界，里面包含：

- Conversation；
- Memory；
- Workspace；
- Planner；
- Tool State；
- Permission；
- Execution Metadata；
- Checkpoint；
- Runtime 内部控制信息。

这些信息对 Runtime 很重要，但不代表都适合进入 LLM 的输入。

所以 Part C 的核心问题是：

Runtime 如何从完整的 Runtime State 中，选择、组织、压缩并组装出 LLM 当前真正需要看到的世界？

Part C 的核心结论是：

Context Builder 是 Runtime 的认知调度中心。它负责把 Runtime State 这个完整世界，经过 Projection、Selection、Retrieval、Compression、Token Budget 和 Message Assembly，转换成 LLM 当前能够理解和推理的 Provider Messages。

可以这样串起来：

Part A:
Prompt 不是 Context

Part B-01:
Context 来自 Runtime State 的 Projection

Part B-02:

Runtime State 本身有完整生命周期

Part C:

Context Builder 把 Runtime State 投影成 Provider Messages

Part D:

Context Window Management 处理有限上下文空间下的工业级取舍

Part E:

Provider Adapter 把统一 Provider Messages 转换成不同模型供应商协议

目录

为什么 Runtime State 不能直接发送给 LLM

Context Projection 是什么

Context Builder 在 Runtime 中的位置

Context Builder Architecture

Context Selection

Conversation Context

Memory Context

Workspace Context

Tool Context

Token Budget Management

Message Assembly

从 Runtime State 到 LLM Intelligence

Context Builder 的几个边界

工业级问答补充

工业级实现 (Industrial Notes)

Context Builder 全景架构图

Runtime 五大模块关系

固定收尾栏目

下一节学习计划

写书 TODO

写书素材

知识地图

面试视角

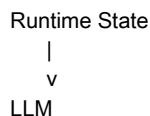
前置问题回收

本 Part 核心认知升级

本章思考题

1. 为什么 Runtime State 不能直接发送给 LLM

最直觉的错误设计是：



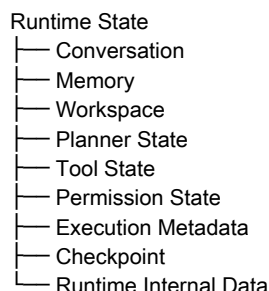
也就是把 Runtime State 整个塞进模型。

这在简单 Demo 里看起来可行，但在 Agent Runtime 中很快会崩溃。

1.1 Runtime State 太完整

Runtime State 是 Runtime 对当前世界的完整认知。

它包含用户目标，也包含大量 LLM 当前不需要知道的内部状态：



完整不等于适合推理。

LLM 当前需要的不是“整个世界”，而是“当前任务相关的世界切片”。

1.2 直接发送会造成信息污染

如果用户当前目标是：

帮我修复登录接口的 bug

Runtime State 中可能还有：

- 过去的 UI 设计讨论；
- 上一次关于数据库迁移的记录；
- 与当前任务无关的 Memory；
- 其他工具执行结果；
- 失败重试的内部日志；
- 大量不相关文件内容。

这些信息进入 LLM 后，会增加推理噪声。

LLM 不是看得越多就越聪明，而是需要看到当前任务最相关的信息。

1.3 直接发送会泄露 Runtime 内部机制

Runtime State 里可能包含：

- 内部策略；
- 权限判断；
- 工具调用细节；

- 执行计划草稿；
- 调试信息；
- 用户不应看到的系统信息。

这些内容不应该直接暴露给模型。

Context Builder 必须把 Runtime State 中的内部机制和模型可见信息分开。

1.4 直接发送无法控制 Token

Runtime State 可能远远超过模型上下文窗口。

即使模型支持 1M token，也不代表 Runtime 可以把所有历史对话、所有文件、所有 Memory、所有 Tool Result 全部发送进去。

Context Window 是一次模型调用的输入输出预算，不是整个 Conversation 的无限仓库。

所以 Runtime 需要：

- Selection；
- Compression；
- Summarization；
- Token Budget；
- Overflow Recovery；
- Fallback Strategy。

1.5 不同任务需要不同 Context

同一个 Runtime State，可以生成多个不同 Context。

例如同一个代码仓库状态：

Runtime State:
完整代码仓库 + 历史对话 + Memory + Tool Results

如果任务是 Debug，Context 可能选择：

错误日志
相关文件
最近失败的测试
相关代码调用链

如果任务是 Code Review，Context 可能选择：

diff
相关规范
历史 review 经验
风险点

如果任务是架构设计，Context 可能选择：

模块边界
数据流
设计约束
历史决策

这说明：

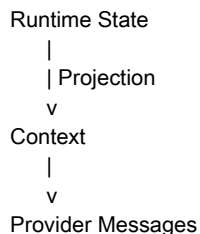
Context 不是 Runtime State 的完整拷贝，而是 Runtime State 面向当前任务的一次 Projection。

2. Context Projection 是什么

Projection 可以理解为：

从一个完整对象中，按照目标和策略，投影出某个视角下需要看到的部分。

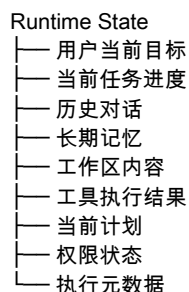
在 Agent Runtime 中：



Context Projection 不是简单过滤字段，而是一次面向任务的认知转换。

2.1 Runtime State 是完整世界

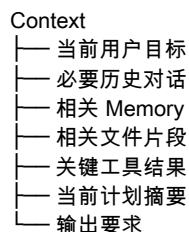
Runtime State 描述 Runtime 当前认为世界是什么样：



这是 Runtime 的内部世界。

2.2 Context 是 LLM 可见世界

Context 描述 LLM 当前应该看到什么：



这是 Runtime 投影给 LLM 的外部视图。

2.3 Provider Messages 是最终模型输入

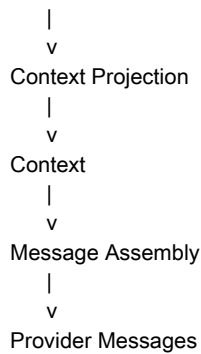
Context 还不是最终请求。

Context Builder 最后会把 Context 组装成 Provider Messages：

```
[
  { role: "system", content: "..."},
  { role: "user", content: "..."},
  { role: "assistant", content: "..."},
  { role: "tool", content: "..."}
]
```

更准确地说：

Runtime State



2.4 Projection 由 Goal、Policy、State 共同决定

Projection 不是随意选择。

它通常由三类输入决定：

Runtime State + Goal + Context Policy

其中：

- Runtime State 提供完整候选信息；
- Goal 决定当前任务要解决什么；
- Context Policy 决定选择、压缩、排序和预算策略。

公式可以写成：

Context = Project(RuntimeState, Goal, ContextPolicy)

Provider Messages 则是：

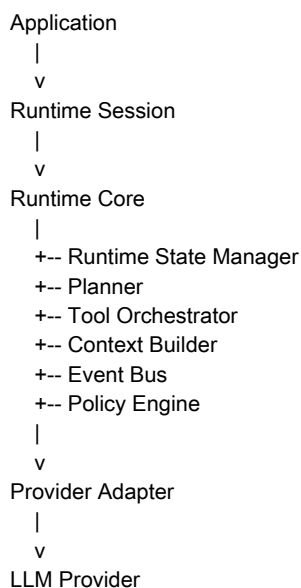
ProviderMessages = Assemble(Context, ProviderAgnosticMessagePolicy)

3. Context Builder 在 Runtime 中的位置

Context Builder 属于 Runtime Core。

它不属于 Provider Adapter，也不属于 Tool 层。

3.1 正确分层



Context Builder 放在 Runtime Core 中，因为它处理的是 Runtime 的核心认知问题：

当前这轮 LLM 应该看到什么？

3.2 为什么不能放 Provider Adapter

Provider Adapter 的职责是协议适配。

它回答的是：

如何把 Runtime 的统一消息格式，转换成某个模型供应商要求的请求格式？

例如：

OpenAI Adapter
Anthropic Adapter
Gemini Adapter
Local Model Adapter

如果把 Context Builder 放进 OpenAI Adapter，会导致：

- Context 策略绑定某个模型供应商；
- 换模型时要重写上下文逻辑；
- Adapter 中出现大量 if/else；
- Runtime Core 无法保持统一认知层；
- 元数据和模型协议耦合在一起。

正确边界是：

Context Builder:
决定发送什么

Provider Adapter:
决定如何发送

3.3 为什么不能放 Tool 层

Tool 的职责是改变世界。

Tool 读取文件、执行命令、查询数据、调用 API。

Tool Result 进入 Runtime 后，应该先更新 Runtime State：

```
Tool Result
|
v
Runtime State Update
|
v
Context Builder
|
v
Provider Messages
```

Tool 不应该直接把结果塞给 LLM。

原因是：

- Tool Result 可能很长；
- Tool Result 可能包含内部细节；
- Tool Result 可能与当前下一步无关；
- Tool Result 需要摘要、提取和重组；

- Tool Result 需要和 Conversation、Memory、Workspace 共同排序。

3.4 Context Builder 应该尽量是纯函数

理想情况下：

```
Context = ContextBuilder.build(RuntimeState, Goal, Policy)
```

Context Builder 不直接修改 Runtime State。

它读取 Runtime State，生成 Context。

这样更容易：

- 测试；
- 调试；
- 重放；
- 对比不同策略；
- 做 explainability；
- 做 A/B 实验。

当然工业系统里可能会记录 metrics、cache 或 trace，但核心语义应该保持：

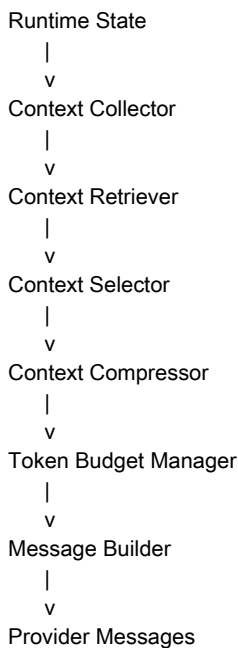
Context Builder 是 Projection，不是 State Mutation。

4. Context Builder Architecture

Context Builder 不是一个简单函数，也不是一个 Prompt Template。

它更像一条 Pipeline。

4.1 总体 Pipeline



每一层职责不同。

4.2 Context Collector

Collector 负责收集候选信息。

候选来源包括：

- Conversation ;
- Memory ;
- Workspace ;
- Planner State ;
- Tool State ;
- User Profile ;
- System Policy ;
- Current Goal ;
- Execution Metadata。

Collector 不负责最终判断哪些必须进入上下文。

它的职责是：

把可能有用的信息收集成候选集。

4.3 Context Retriever

Retriever 负责从大规模候选空间中找相关信息。

它可能使用：

- keyword search ;
- vector search ;
- hybrid search ;
- symbol index ;
- AST index ;
- call graph ;
- git history ;
- file path hint ;
- recency index。

Retriever 的职责是：

发现可能相关的信息。

但发现不等于一定发送。

4.4 Context Selector

Selector 负责决定哪些信息进入本轮 Context。

它会考虑：

- 当前 Goal ;
- 当前 Planner Step ;
- 语义相关性 ;

- 时间新鲜度；
- 重要性；
- 用户显式指令；
- 安全策略；
- token 成本；
- 信息可信度。

Selector 是 Context Builder 最核心的能力。

它决定 LLM 当前“看见什么”。

4.5 Context Compressor

Compressor 负责把长信息变短。

常见方式：

- summary compression；
- structured compression；
- extractive compression；
- sliding window；
- diff compression；
- tool result extraction；
- conversation summary。

例如一段很长的工具输出：

测试失败日志 5000 行

进入 Context 前可能变成：

测试失败：login.test.ts 第 42 行断言失败。
预期 status=200，实际 status=401。
相关错误：JWT expired。

4.6 Token Budget Manager

Token Budget Manager 负责分配有限上下文空间。

例如：

Total Budget: 128k

System Policy:	4k
Current Goal:	2k
Recent Messages:	16k
Memory:	8k
Workspace:	64k
Tool Results:	16k
Output Reserve:	18k

它不是单纯裁剪，而是在不同信息源之间做预算分配。

4.7 Message Builder

Message Builder 负责把 Context 组装成模型输入。

它会生成：

- system message ;
- developer message ;
- user message ;
- assistant history ;
- tool result message ;
- structured context blocks ;
- output instruction ;
- safety and policy message。

这一步输出的是 Provider-Agnostic Messages。

也就是说，它还没有绑定 OpenAI、Anthropic 或其他模型供应商。

5. Context Selection

Context Selection 是 Runtime 决定 LLM 看到什么的核心能力。

它不是简单 top-k 检索，而是组合策略。

5.1 Selection 的本质

Selection 可以写成：

```
selected = select(
    candidates,
    goal,
    plannerState,
    contextPolicy,
    tokenBudget
)
```

它的本质是：

从 Runtime State 的候选信息中，选择当前推理最相关、最必要、最安全、最符合预算的信息。

5.2 Goal Driven Selection

当前目标决定主要信息范围。

如果目标是：

修复登录 bug

更应该选择：

- login 相关代码；
- auth 相关配置；
- 最近错误日志；
- 相关测试；
- 用户关于登录的要求。

不应该优先选择：

- UI 色彩方案；

- 旧的部署讨论；
- 无关模块代码。

5.3 Planner Driven Selection

Planner State 决定当前执行步骤。

例如：

Plan:

1. 复现错误
2. 定位代码
3. 修改实现
4. 运行测试
5. 总结结果

如果当前在第 2 步，Context Builder 更应该选择：

- 错误日志；
- 相关代码；
- 调用链；
- 文件结构。

如果当前在第 4 步，Context Builder 更应该选择：

- 测试命令；
- 测试结果；
- 最近修改 diff。

5.4 Recency

近期消息通常更重要。

但 Recency 不能单独决定一切。

例如 1 小时前用户说：

这个项目不能引入新依赖。

虽然不是最近消息，但它是重要约束，必须保留。

所以 Recency 要和 Importance 结合。

5.5 Semantic Retrieval

语义检索用于找到“意思相关”的内容。

例如用户说：

退款失败

代码里可能没有 refund 这个文件名，而是：

```
reverseTransaction  
cancelPayment  
paymentReversal
```

关键词搜索可能找不到，但 Embedding 可以发现语义接近。

5.6 Selection 是组合策略

工业级 Selection 往往综合多种信号：

```
Score =  
  goal_relevance  
+ semantic_similarity  
+ recency  
+ importance  
+ user_pin  
+ planner_need  
- token_cost  
- security_risk  
- stale_penalty
```

因此 Context Selection 不是一个固定算法，而是一套策略系统。

6. Conversation Context

Conversation Context 解决的是：

历史对话如何进入当前 Context？

Conversation 是发生过什么，Context 是当前应该看到什么。

6.1 Conversation 不能无限增长

如果每轮都把完整聊天记录发给模型，会出现：

- token 爆炸；
- 成本增加；
- 速度变慢；
- 推理噪声增加；
- 旧信息污染当前任务；
- 关键约束被大量闲聊淹没。

所以 Conversation 必须经过 Context Builder。

6.2 Conversation Context 的三层结构

```
Conversation Context  
├── Raw Messages  
├── Recent Window  
└── Summary Memory
```

Raw Messages 是原始聊天记录。

Recent Window 是最近若干轮重要对话。

Summary Memory 是对历史上下文的摘要。

6.3 Sliding Window

Sliding Window 保留最近若干消息。

优点是简单、稳定、低成本。

缺点是可能丢失早期重要约束。

6.4 Importance Ranking

Importance Ranking 会把重要消息提权。

例如：

- 用户明确约束；
- 项目目标；
- 架构决策；
- 用户偏好；
- 失败原因；
- 已确认结论。

这些消息即使不新，也可能进入 Context。

6.5 Conversation Summary

Summary 用来压缩长对话。

它保存的是：

- 已完成什么；
- 当前任务状态；
- 重要约束；
- 用户偏好；
- 待回答问题；
- 后续计划。

Summary 不是 Memory。

Conversation Summary 更偏当前会话连续性。

Memory 更偏跨会话长期价值。

7. Memory Context

Memory Context 解决的是：

长期记忆如何进入当前 Context？

Memory 是 Agent 的长期认知资产，但并不是所有 Memory 都应该每次进入 Context。

7.1 Memory 与 Conversation 的区别

Conversation 记录当前会话发生过什么。

Memory 保存跨会话仍然有价值的信息。

例如：

Conversation:

今天讨论了 Context Builder 的章节结构。

Memory:

用户学习 Agent Runtime 时希望每个 Part 结束后加入固定收尾栏目。

7.2 Memory 不能全部发送

Memory 可能不断增长。

如果全部发送：

- token 会爆炸；
- 旧偏好可能污染当前任务；
- 不相关 Memory 增加噪声；
- 过时 Memory 可能误导模型。

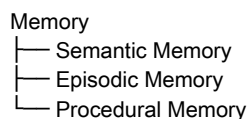
所以 Memory 需要 Retrieval 和 Selection。

7.3 Memory Retrieval

Memory Retrieval 的常见方式：

- 关键词匹配；
- 语义检索；
- hybrid retrieval；
- 时间衰减；
- 用户显式标签；
- 任务类型匹配；
- importance ranking。

7.4 Memory 分类



Semantic Memory 保存事实和偏好。

Episodic Memory 保存事件和经历。

Procedural Memory 保存做事方式和流程偏好。

本次学习中固化的固定收尾栏目，更接近 Procedural Memory。

7.5 Memory 如何知道保存什么

Memory 写入通常有两种来源。

第一种是显式 Memory：

请记住：以后每个 Part 结束都加工业级实现栏目。

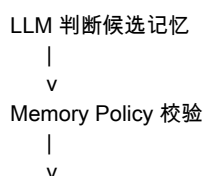
这种情况可以由 Runtime 明确触发 Memory Write。

第二种是隐式 Memory：

用户反复强调按 README 主线推进。

这种情况通常由 LLM 或专门的 Memory Extractor 判断其长期价值，再交给 Memory Tool 或 Memory Store 保存。

更工业化的设计是：



8. Workspace Context

Workspace Context 解决的是：

Agent 如何理解外部世界，例如代码、文件、知识库和项目结构？

对于 Coding Agent 来说，Workspace 是非常核心的 Context 来源。

8.1 Workspace 是 Runtime State 的一部分

Workspace 可能包含：

- 文件树；
- 文件内容；
- package 信息；
- git diff；
- 测试结果；
- 代码符号；
- 依赖关系；
- 构建配置；
- 文档；
- issue 或 PR 信息。

这些都属于 Runtime 对外部世界的认知。

8.2 不能把整个 Workspace 发送给 LLM

大型项目可能有：

- 几千个文件；
- 几百万行代码；
- 多层组件；
- 大量依赖；
- 多语言模块。

全部发送不现实，也没有必要。

Context Builder 必须决定：

当前任务需要哪些文件？
需要完整文件还是片段？
需要代码还是摘要？
需要调用链还是 diff？

8.3 文件命名很差时 Retriever 如何找到文件

不是只靠文件名。

工业级 Workspace Retrieval 会结合多层信息：

File Name
Content Embedding

Symbol Index
AST
Call Graph
Import Graph
Git History
Runtime Error Stack
User Hints

如果文件名很差，例如：

utils2.ts
abc.ts
handler_old.ts

Retriever 仍然可以通过内容语义、函数名、调用关系、错误栈和最近修改记录找到相关文件。

8.4 代码转向量如何理解

Embedding 是把文本映射成一组数字。

例如代码片段：

```
function refundOrder(orderId) {  
  return paymentService.reverse(orderId)  
}
```

Embedding 后可能变成：

[0.23, 0.65, 0.11, ...]

这些数字不是给人读的，而是给向量检索系统比较语义距离的。

如果另一个片段表达了“取消支付”或“反向交易”，即使用词不同，向量距离也可能很近。

8.5 Embedding 与 AST 的关系

Embedding 擅长语义相似。

AST 擅长结构理解。

例如：

Embedding:
知道 refund、cancelPayment、reverseTransaction 语义接近。

AST:
知道 refund() 调用了 transactionService.reverse()。

工业级代码理解通常需要两者结合。

9. Tool Context

Tool Context 解决的是：

工具执行结果如何进入 LLM 当前输入？

Tool Result 不等于 Tool Context。

9.1 Tool Result 是原始结果

例如工具返回：

Command failed with exit code 1
5000 lines of logs...
stack trace...
environment variables...

internal metadata...

这是原始 Tool Result。

它可能太长、太吵、太内部化。

9.2 Tool Context 是投影后的结果

Context Builder 会把 Tool Result 转换成：

测试失败：

- 文件：login.test.ts
- 断言：expected 200, received 401
- 可能原因：JWT expired
- 下一步建议：检查 auth middleware

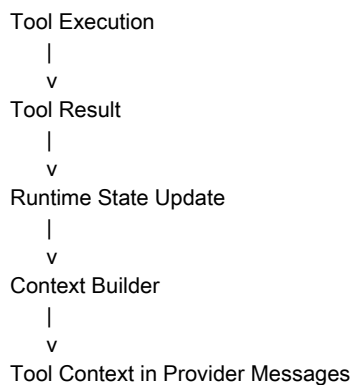
这是 Tool Context。

9.3 Tool Result 为什么不能直接进入 LLM

原因包括：

- 信息噪音；
- token 成本；
- 安全风险；
- Runtime 内部信息泄露；
- 与当前目标无关；
- 缺少结构化摘要。

所以 Tool 的正确流程是：



10. Token Budget Management

Token Budget Management 解决的是：

上下文空间有限时，不同信息源如何分配预算？

10.1 上下文长度 1M 指什么

上下文长度指的是一次模型调用的输入输出总窗口。

它不是当前 Conversation 中所有历史输入输出的总上限。

更准确地说：

Model Context Window
= 当前一次 Provider Request 中

输入 messages + 工具描述 + 其他上下文 + 预留输出的总 token 空间

所以即使一个模型支持 1M token，Runtime 仍然需要 Context Builder。

10.2 为什么 1M 仍然不够

原因是：

- 真实 Workspace 可以超过 1M；
- 长期运行 Agent 会积累大量事件；
- Memory 可能持续增长；
- 工具输出可能非常长；
- 输出也需要预留 token；
- 模型看太多会增加推理噪声；
- 成本和延迟会显著上升。

大窗口降低了压力，但没有取消 Context Engineering。

10.3 超限怎么办

如果发现输入过大，不一定直接终止。

工业级 Runtime 通常会尝试：

1. Drop low-priority context
2. Compress long blocks
3. Summarize conversation
4. Reduce workspace snippets
5. Keep critical constraints
6. Reserve output tokens
7. Retry assembly
8. If still overflow, ask user or fail gracefully

10.4 Selection 与 Budget 的区别

Selection 解决：

哪些信息值得进入 Context？

Budget 解决：

这些信息在有限窗口里各占多少空间？

可以类比：

Selection:
决定邀请谁参加会议。

Budget:
决定每个人发言多久。

它们会互相影响，但职责不同。

11. Message Assembly

Message Assembly 是 Context Builder 的最后一步。

它把筛选、检索、压缩、预算后的 Context 组装成最终 LLM Input。

11.1 Message Assembly 的输入

输入可能包括：

- Selected Conversation
- Selected Memory
- Selected Workspace
- Selected Tool Context
- Current Goal
- Planner State
- System Policy
- Output Requirements

11.2 Message Assembly 的输出

输出是 Provider-Agnostic Messages：

- Provider Messages
 - System / Developer Instructions
 - Current User Goal
 - Recent Conversation
 - Relevant Memory
 - Workspace Context
 - Tool Context
 - Current Plan
 - Output Contract

这些 Messages 还不是某个供应商特定请求。

Provider Adapter 会在下一层完成协议转换。

11.3 Message Assembly 的价值

Message Assembly 不是字符串拼接。

它要保证：

- 顺序合理；
- 角色正确；
- 约束明确；
- 信息结构清晰；
- 高优先级信息不被淹没；
- 输出要求与当前任务一致；
- Provider Adapter 可以稳定消费。

12. 从 Runtime State 到 LLM Intelligence

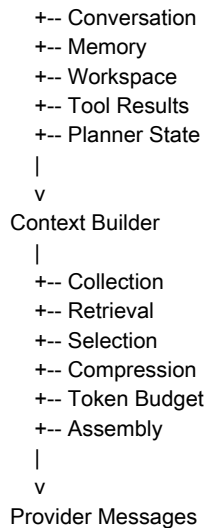
经过 Part C，可以重新理解一次 Agent 调用过程。

Step 1：Runtime 接收任务

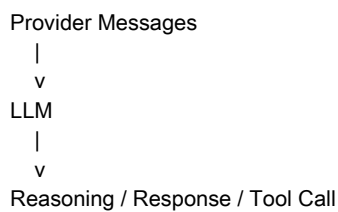
```
graph TD
    A[User Goal] --> B[Runtime State Update]
```

Step 2：Context Builder 构造认知

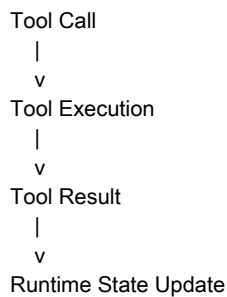
```
graph TD
    A[Runtime State] --> B[ ]
```



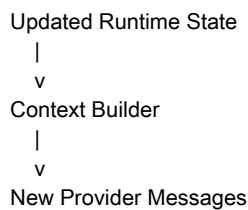
Step 3 : LLM 推理



Step 4 : Tool 改变世界



Step 5 : 下一轮重新 Projection



这就是 Agent Loop 的认知闭环。

13. Context Builder 的几个边界

13.1 Context Builder 不是 Prompt Template

Prompt Template 是静态模板。

Context Builder 是动态系统。

Prompt Template 更像：

把变量填进固定文本。

Context Builder 更像：

根据 Runtime State、Goal 和 Policy，动态构造 LLM 当前可见世界。

13.2 Context Builder 不是 Search

Retriever 负责发现信息。

Context Builder 负责决定认知。

可以这样区分：

Search / Retriever:

这里可能有什么相关信息？

Context Builder:

当前 LLM 应该看到哪些信息，以什么形式看到？

13.3 Context Engineering 不是 Prompt Engineering

Prompt Engineering 关注：

如何写好一段 Prompt？

Context Engineering 关注：

如何构建模型当前看到的世界？

Prompt 是结果。

Context Builder 是生成结果的工程系统。

13.4 Context Builder 与 Provider Adapter 的边界

Context Builder:

What to send

Provider Adapter:

How to send

Context Builder 属于 Runtime 的认知层。

Provider Adapter 属于模型协议适配层。

14. 工业级问答补充

14.1 Context Builder 的 Policy 是不是业务策略

是，但不只是业务策略。

Context Policy 包含：

- 任务类型策略；
- 产品形态策略；
- 安全策略；
- 成本策略；
- token 策略；
- Memory 策略；
- Workspace 策略；
- Tool Result 策略。

它可以承载业务形态中的抽象化逻辑。

例如客服 Agent、Coding Agent、Research Agent 的 Context Policy 一定不同。

14.2 Context Builder 为什么必须属于 Runtime Core

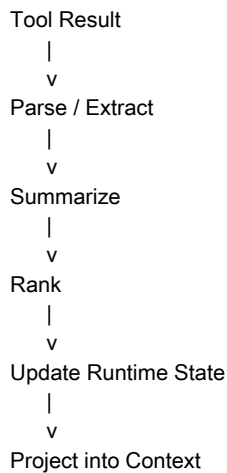
因为它不应该绑定具体模型。

无论后面接 OpenAI、Anthropic、Gemini 还是本地模型，Runtime 都应该先构造统一的 Provider-Agnostic Messages，再由 Adapter 转换。

否则模型适配层会变成混乱的 if/else。

14.3 Tool Result 如何转 Tool Context

通常需要：



Tool Context 是工具结果的可推理视图。

14.4 Memory Retrieval 如何排序

可能综合：

- semantic similarity ;
- importance ;
- recency ;
- frequency ;
- user explicitness ;
- task type ;
- conflict detection ;
- decay。

Memory 不是越多越好，而是越相关越好。

14.5 Agent 的核心不是知道，而是知道什么时候需要知道什么

Agent 不可能把所有东西都放进当前上下文。

真正的能力在于：

- 什么时候需要读文件？
- 什么时候需要召回 Memory？
- 什么时候需要查工具结果？

什么时候应该压缩历史？
什么时候应该向用户确认？

这就是 Context Builder 的认知控制价值。

15. 工业级实现 (Industrial Notes)

从 Part C 开始，每个 Part 结束时都应该加入工业级实现视角。

15.1 工业系统如何实现 Context Builder

一个工业级 Context Builder 通常不是单个类，而是一组协作模块：

```
ContextBuilder
├── Collectors
├── Retrievers
├── Selectors
├── Compressors
├── BudgetManager
├── MessageAssembler
├── PolicyEngine
├── TraceLogger
└── Evaluation Hooks
```

15.2 理论概念到工程模块映射

```
Context Projection -> buildContext()
Context Selection  -> selector.rank()
Memory Retrieval   -> memoryRetriever.search()
Workspace Context  -> workspaceIndexer.lookup()
Token Budget       -> budgetManager.allocate()
Message Assembly   -> messageAssembler.build()
Provider Adapter   -> providerAdapter.convert()
```

15.3 工业级取舍

常见取舍包括：

- 准确性 vs 成本；
- 更多 Context vs 更低噪声；
- 长窗口 vs 快响应；
- 自动检索 vs 用户显式选择；
- 摘要压缩 vs 原文保真；
- 通用策略 vs 业务定制；
- 纯函数可测试性 vs Runtime cache 性能。

15.4 可观测性

Context Builder 必须可观测。

否则当 Agent 回答错误时，很难知道是：

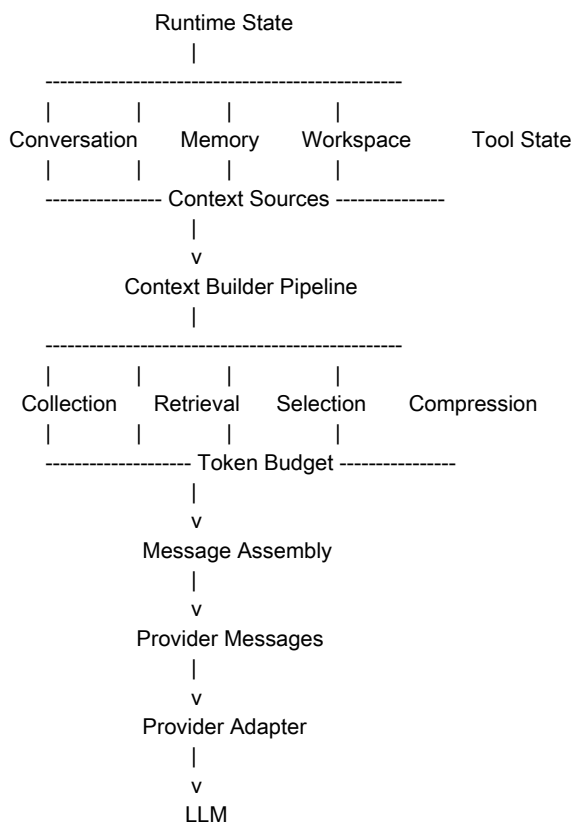
- LLM 推理错了；
- Context Selection 漏了；
- Retriever 找错了；
- Memory 召回错了；

- Token Budget 裁掉了关键内容；
- Provider Adapter 转换出错了。

建议记录：

候选信息数量
被选中信息
被丢弃信息及原因
压缩前后长度
token 预算分配
最终 messages 摘要
策略版本

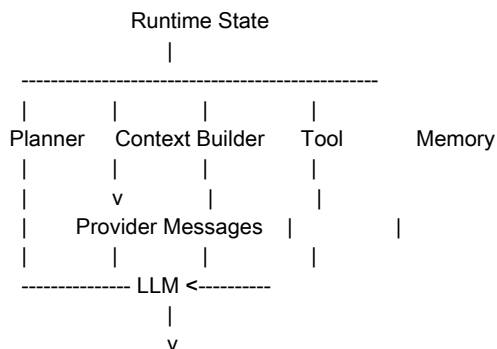
16. Context Builder 全景架构图



这个图表达了一个核心事实：

LLM 看到的世界，不是 Runtime State 本身，而是 Context Builder 投影出来的世界。

17. Runtime 五大模块关系



可以这样理解：

- Planner 决定下一步要做什么；
- Context Builder 决定 LLM 当前看到什么；
- LLM 负责推理和生成行动；
- Tool 改变外部世界并返回结果；
- Memory 保存跨会话长期价值；
- Runtime State 串联所有模块。

Context Builder 是这些模块之间的认知枢纽。

18. 固定收尾栏目

今天的学习中确定了一个后续学习规则：

当一个章节、Part 或学习阶段接近结束时，正式笔记应加入固定收尾栏目。

固定栏目包括：

下一节学习计划

写书 TODO

写书素材

本 Part 核心认知升级

工业级实现 (Industrial Notes)

知识地图 (Knowledge Map)

面试视角

前置问题回收 (Pending Questions)

其中“前置问题回收”用于记录当前章节出现但属于未来章节的问题，后续进入对应章节时主动回收。

19. 下一节学习计划

下一节进入：

Day04 Part D:
Context Window Management

Part D 要解决：

Context Window 有限时，工业级 Runtime 如何管理上下文空间？

重点问题：

为什么模型支持 1M token 后，Context Engineering 仍然重要？

Context Overflow 如何检测和恢复？

Token Budget Manager 如何动态调整？

Long-running Agent 如何运行数小时甚至数天？

历史对话、Memory、Workspace、Tool Result 如何竞争上下文预算？

20. 写书 TODO

TODO 1：把 Context Builder 作为“认知层”独立成章

不要只把它写成 Prompt 拼接模块。

更好的表述是：

Context Builder 是 Runtime 的认知调度中心。

TODO 2：补一张 Runtime State 到 Provider Messages 的完整图

这张图应贯穿 Part A、B、C：

Runtime State -> Context Projection -> Context -> Provider Messages -> LLM

TODO 3：增加“前端视角理解 Context Builder”

可以类比：

React State -> Render -> DOM

Runtime State -> Context Builder -> Provider Messages

Redux Selector 也可以类比 Context Selection。

TODO 4：增加 Context Builder 与 Provider Adapter 边界

要明确：

Context Builder 决定发送什么。

Provider Adapter 决定如何发送。

TODO 5：增加 Context Builder 的可观测性专题

工业级 Agent Debug 很依赖 Context Trace。

需要解释：

- 为什么选了这些上下文；
- 为什么丢弃那些上下文；
- 为什么某条 Memory 进入了 Context；
- 为什么某个文件没有进入 Context；
- token 是如何分配的。

21. 写书素材

素材一：

Prompt 是 Context Builder 的产物，不是 Runtime 的起点。

素材二：

Runtime State 是 Agent 的完整世界，Context 是 LLM 当前看到的世界。

素材三：

Context Builder 是 Agent Runtime 的认知调度中心，它负责从 Runtime State 中选择、组织、压缩并构造当前任务最需要的信息。

素材四：

Agent 的智能并不是来自 LLM 看到了全部世界，而是来自 Runtime 通过 Context Builder，为 LLM 构建了一个足够真实、足够相关、又足够精简的世界。

素材五：

Retriever 负责发现信息，Context Builder 负责决定认知。

素材六：

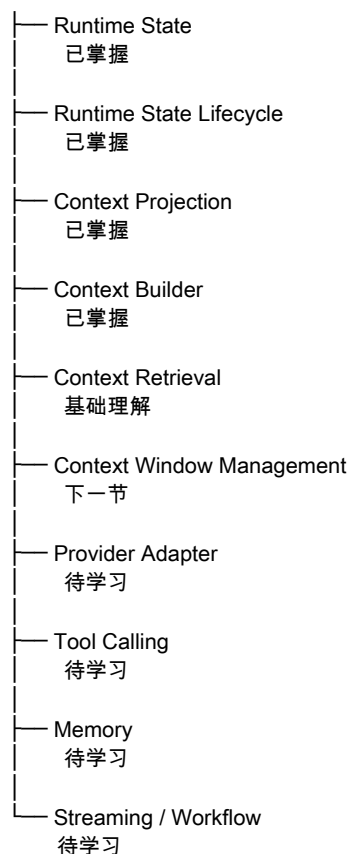
Context Engineering 不是写更长的 Prompt，而是构建更合适的模型可见世界。

素材七：

Agent 的核心不是知道所有东西，而是知道什么时候需要知道什么。

22. 知识地图

Agent Runtime



23. 面试视角

问题 1：Context Builder 是什么？

Context Builder 是 Agent Runtime 中负责构建 LLM 输入的模块。它从 Runtime State 中收集 Conversation、Memory、Workspace、Tool State 等信息，通过 Selection、Retrieval、Compression、Token Budget 和 Message Assembly，最终生成发送给 LLM 的 Provider

Messages。它的核心目标不是提供最多的信息，而是提供当前任务最相关的信息。

问题 2：为什么 Runtime State 不能直接发送给 LLM？

Runtime State 是 Agent 的完整世界，而 LLM 每一轮推理只需要其中的一部分。如果全部发送，会浪费 token、增加成本、引入噪声、暴露内部机制，还可能降低推理质量。因此需要 Context Builder 对 Runtime State 做面向当前任务的 Projection。

问题 3：Context Builder 和 Prompt Engineering 有什么区别？

Prompt Engineering 更关注如何写一段 Prompt。Context Builder 更关注如何动态构建模型当前看到的整个输入世界。Prompt 是最终文本表现之一，Context Builder 是生成这些文本和消息的工程系统。

问题 4：Context Builder 和 Provider Adapter 的边界是什么？

Context Builder 负责决定发送什么；Provider Adapter 负责决定如何发送。前者属于 Runtime Core 的认知层，后者属于模型协议适配层。

问题 5：Context Selection 和 Token Budget Management 的区别是什么？

Context Selection 决定哪些信息值得进入 Context。Token Budget Management 决定这些信息在有限上下文空间中如何分配预算。Selection 更关注相关性和必要性，Budget 更关注空间分配和溢出处理。

24. 前置问题回收

本 Part 留下的问题要在后续章节主动回收。

Day04 Part D：Context Window Management

Context 超限后如何自动恢复？

Token Budget Manager 如何动态调整？

Context Selection 和 Token Budget Management 的边界如何在实现中落地？

1M token 为什么仍然不等于无限上下文？

Day04 Part E：Provider Adapter

Context Builder 与 Provider Adapter 的边界如何体现在代码接口上？

不同模型供应商的 message schema 如何统一？

Tool Calling Adapter 和 Message Adapter 如何协同？

Day05：Tool Calling

Tool Result 如何转化为 Tool Context？

Tool Result 是否应该覆盖旧 State？

Tool 执行失败如何进入 Runtime State 和下一轮 Context？

Day06：Memory

Memory 如何判断保存什么？

Memory Retrieval 如何排序？

Memory 生命周期如何管理？

显式 Memory 与隐式 Memory 如何区分？

25. 本 Part 核心认知升级

第一层：从 Prompt 到 Context

过去理解：

User Prompt -> LLM -> Answer

现在理解：

Runtime State -> Context Builder -> Provider Messages -> LLM

第二层：从拼接文本到认知 Projection

过去认为：

Context Builder = 拼 Prompt

现在理解：

Context Builder = Runtime 的认知调度中心

第三层：从静态输入到动态世界

过去认为：

LLM 输入是一段文本。

现在理解：

LLM 输入是 Runtime 根据当前 State、Goal 和 Policy 动态生成的世界视图。

第四层：从“看更多”到“看对”

过去认为：

模型看到越多越好。

现在理解：

模型需要看到当前任务最相关、最可靠、最精简的信息。

最终认知

Context Builder 是 Agent Runtime 的认知中枢。它负责把 Runtime State 这个无限复杂的世界，经过选择、检索、压缩和组织，转换成 LLM 当前能够理解和推理的有限世界。

26. 本章思考题

为什么 Runtime State 越完整，反而越不能直接发送给 LLM？

Context 和 Runtime State 最大区别是什么？

为什么同一个 Runtime State 可以生成多个 Context？

React 的 State -> Render -> DOM 和 Runtime State -> Context Builder -> Provider Messages 有什么对应关系？

为什么 Context Builder 必须属于 Runtime Core，而不是 Provider Adapter？

Tool Result 为什么不能直接进入 LLM ?

Context Selection 和 Token Budget Management 的职责边界是什么 ?

Memory 为什么需要 Retrieval 和 Selection 后才能进入 Context ?

文件命名很差时 , Workspace Retriever 还能依赖哪些信息找到相关文件 ?

如果来实现一个最小 Agent Runtime , Context Builder 的第一版应该包含哪些模块 ?

来源

- ChatGPT 分享学习记录 : <https://chatgpt.com/share/6a5f3b3e-cd80-83e8-856e-d7aaa7990a6b>

- 本地源记录 : [source/day04-part-c-chatgpt-share-source.md](#)